In [3]:
```
'''
Ass01
07-02-2026

'''

import re

def check_password(password):

    if len(password) < 8:
        return "Password must be at least 8 characters long."

    if not re.search("[A-Z]", password):
        return "Password must contain at least one uppercase letter."

    if not re.search("[a-z]", password):
        return "Password must contain at least one lowercase letter."

    if not re.search("[0-9]", password):
        return "Password must contain at least one number."

    if not re.search("[@#$%^&*!]", password):
        return "Password must contain at least one special character (@#$%^&*! )."

    return "Strong Password!"

password = input("Enter your password: ")

result = check_password(password)

print(result)
```

```
Strong Password!
```

In [ ]:
```
'''
Ass02
09/02/2026

1. Voice Assistants (Google Assistant, Siri)
AI understands voice commands using speech recognition and machine learning models.

2. Recommendation Systems (YouTube, Netflix)
Machine learning analyzes user watch history to recommend relevant content.

3. Email Spam Filtering (Gmail)
ML models classify emails as spam or important based on patterns.

4. Maps and Navigation (Google Maps)
AI analyzes traffic data and suggests the fastest route.

5. Face Unlock in Smartphones
Machine learning analyzes facial features to verify user identity.

6. Online Shopping Recommendations (Amazon)
```

```
AI recommends products based on browsing and purchase history.

7. Social Media Feed Personalization (Instagram, Facebook)
ML algorithms show posts based on user interactions.

8. Chatbots
AI chatbots use natural language processing to answer user questions.

9. Autocorrect and Text Prediction (Gboard)
ML predicts words and corrects spelling while typing.

10. Fraud Detection in Banking
ML analyzes transaction patterns to detect suspicious activity.

'''
```

In [4]:

```
'''
Ass03
10-02-2026
'''
# Smart Input Program


name = input("Enter your name: ")
age = int(input("Enter your age: "))
hobby = input("Enter your hobby: ")


if age < 13:
    category = "Child"
elif age < 20:
    category = "Teenager"
elif age < 60:
    category = "Adult"
else:
    category = "Senior Citizen"


print("\nHello", name + "!")
print("You are", age, "years old and belong to the", category, "category.")
print("It's great that you enjoy", hobby + "!")
```

```
Hello gowtham!
You are 22 years old and belong to the Adult category.
It's great that you enjoy playing cricket!
```

In [5]:

```
'''
Ass04
17-02-2026
'''
# Function to run FizzBuzz logic and count occurrences

def fizz_buzz():

    fizz_count = 0
    buzz_count = 0
```

```
    fizzbuzz_count = 0

    for num in range(1, 51):

        if num % 3 == 0 and num % 5 == 0:
            print("FizzBuzz")
            fizzbuzz_count += 1

        elif num % 3 == 0:
            print("Fizz")
            fizz_count += 1

        elif num % 5 == 0:
            print("Buzz")
            buzz_count += 1

        else:
            print(num)

    print("\nTotal Counts:")
    print("Fizz:", fizz_count)
    print("Buzz:", buzz_count)
    print("FizzBuzz:", fizzbuzz_count)


fizz_buzz() # Calling the function
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz

Total Counts:
Fizz: 13
Buzz: 7
FizzBuzz: 3
```

In [6]:
```
'''
Ass05
19-02-2026
'''
# Student Data Manager

students = {
    "Rahul": 85,
    "Priya": 92,
    "Arjun": 70,
    "Neha": 60,
    "Kiran": 45
}
print("Student Marks:", students)
topper = max(students, key=students.get)
print("Topper:", topper, "-", students[topper])
average = sum(students.values()) / len(students)
print("Class Average:", average)
print("\nGrades:")
for name, marks in students.items():

    if marks >= 90:
        grade = "A"
    elif marks >= 75:
        grade = "B"
    elif marks >= 50:
        grade = "C"
    else:
        grade = "Fail"

    print(name, ":", grade)
```

```
Student Marks: {'Rahul': 85, 'Priya': 92, 'Arjun': 70, 'Neha': 60, 'Kiran': 45}
Topper: Priya - 92
Class Average: 70.4

Grades:
Rahul : B
Priya : A
Arjun : C
Neha : C
Kiran : Fail
```

In [7]:
```
'''
Ass06
20-02-2026
'''
import time
import numpy as np

python_list = list(range(1000000))
numpy_array = np.arange(1000000)

start_time = time.time()
python_result = [x * 2 for x in python_list]
end_time = time.time()
```

```
python_time = end_time - start_time
print("Python List Time:", python_time)

start_time = time.time()
numpy_result = numpy_array * 2
end_time = time.time()
numpy_time = end_time - start_time
print("NumPy Array Time:", numpy_time)
```

```
Python List Time: 0.20716190338134766
NumPy Array Time: 0.006372928619384766
```

In [8]:

```
'''
Ass07
24-02-2026
'''
import pandas as pd
df=pd.read_csv('tested.csv')
print(df)
```

```
     PassengerId  Survived  Pclass  \
0            892         0       3
1            893         1       3
2            894         0       2
3            895         0       3
4            896         1       3
..           ...       ...     ...
413         1305         0       3
414         1306         1       1
415         1307         0       3
416         1308         0       3
417         1309         0       3

                                          Name     Sex   Age  SibSp  Parch  \
0                                Kelly, Mr. James    male  34.5      0      0
1                Wilkes, Mrs. James (Ellen Needs)  female  47.0      1      0
2                       Myles, Mr. Thomas Francis    male  62.0      0      0
3                                Wirz, Mr. Albert    male  27.0      0      0
4    Hirvonen, Mrs. Alexander (Helga E Lindqvist)  female  22.0      1      1
..                                            ...     ...   ...    ...    ...
413                           Spector, Mr. Woolf    male   NaN      0      0
414                  Oliva y Ocana, Dona. Fermina  female  39.0      0      0
415                  Saether, Mr. Simon Sivertsen    male  38.5      0      0
416                           Ware, Mr. Frederick    male   NaN      0      0
417                      Peter, Master. Michael J    male   NaN      1      1

                 Ticket      Fare Cabin Embarked
0                330911    7.8292   NaN        Q
1                363272    7.0000   NaN        S
2                240276    9.6875   NaN        Q
3                315154    8.6625   NaN        S
4               3101298   12.2875   NaN        S
..                  ...       ...   ...      ...
413           A.5. 3236    8.0500   NaN        S
414            PC 17758  108.9000  C105        C
415  SOTON/O.Q. 3101262    7.2500   NaN        S
416              359309    8.0500   NaN        S
417                2668   22.3583   NaN        C

[418 rows x 12 columns]
```

In [9]:

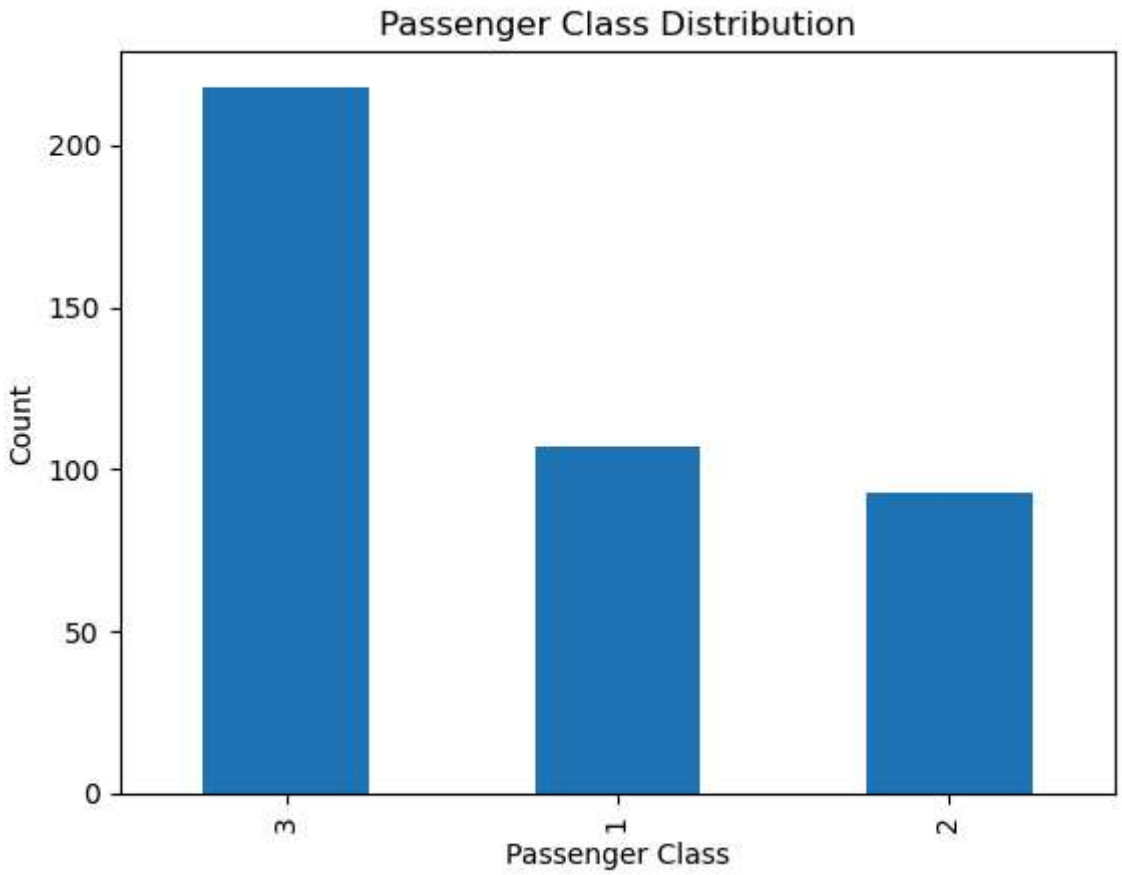
```python
print("Top 5 Rows of the Dataset:")
print(df.head())
print("\nHighest Values in Each Numeric Column:")
print(df.max(numeric_only=True))
print("\nMissing Values in Each Column:")
print(df.isnull().sum())
```

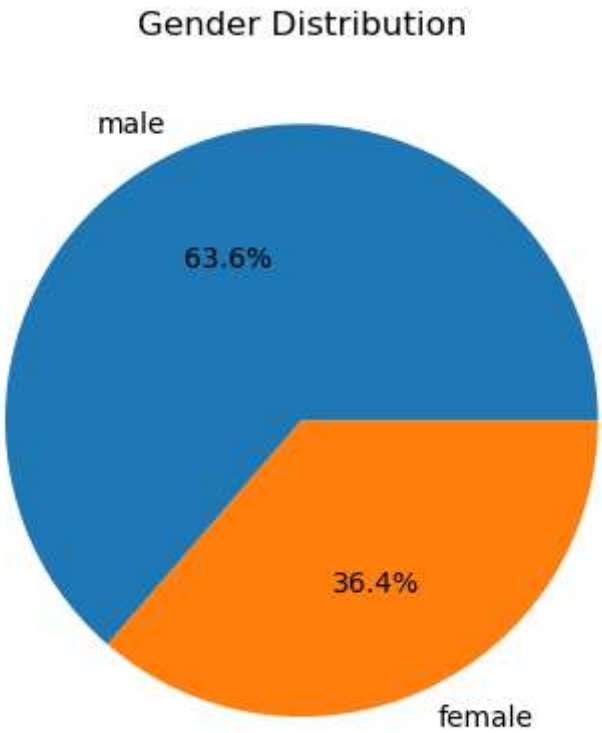
```
Top 5 Rows of the Dataset:
   PassengerId  Survived  Pclass  \
0          892         0       3
1          893         1       3
2          894         0       2
3          895         0       3
4          896         1       3

                                          Name     Sex   Age  SibSp  Parch  \
0                               Kelly, Mr. James    male  34.5      0      0
1               Wilkes, Mrs. James (Ellen Needs)  female  47.0      1      0
2                      Myles, Mr. Thomas Francis    male  62.0      0      0
3                               Wirz, Mr. Albert    male  27.0      0      0
4  Hirvonen, Mrs. Alexander (Helga E Lindqvist)  female  22.0      1      1

    Ticket     Fare Cabin Embarked
0   330911   7.8292   NaN        Q
1   363272   7.0000   NaN        S
2   240276   9.6875   NaN        Q
3   315154   8.6625   NaN        S
4  3101298  12.2875   NaN        S

Highest Values in Each Numeric Column:
PassengerId    1309.0000
Survived          1.0000
Pclass            3.0000
Age              76.0000
SibSp             8.0000
Parch             9.0000
Fare            512.3292
dtype: float64

Missing Values in Each Column:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

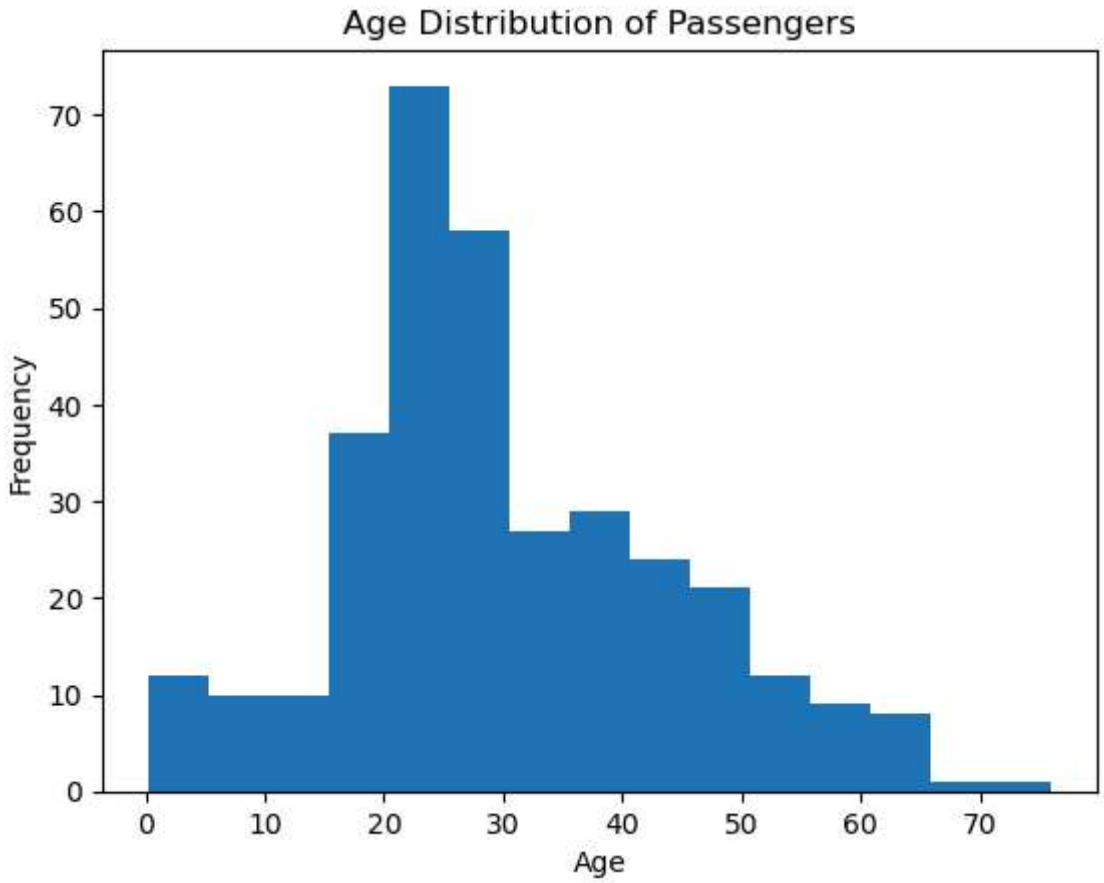
In [2]:

```
'''
ass08
28-02-2026

'''
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('tested.csv')
```

```
df['Pclass'].value_counts().plot(kind='bar')
plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()

df['Sex'].value_counts().plot(kind='pie',autopct='%1.1f%%')
plt.title("Gender Distribution")
plt.ylabel("")
plt.show()

df['Age'].plot(kind='hist', bins=15)
plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```







In [3]:
```
'''

From the graphs, we can understand some basic patterns in the dataset.
The bar chart shows that most of the passengers were in the third class, which mean
```

```
The pie chart shows the number of male and female passengers, and it looks like the
The histogram shows the age distribution of the passengers, and we can see that man
Overall, these graphs help us understand the passenger information in a clear and v

'''
```

Out[3]:
```
'\n\nFrom the graphs, we can understand some basic patterns in the dataset.\nThe b
ar chart shows that most of the passengers were in the third class, which means ma
ny people preferred cheaper travel.\nThe pie chart shows the number of male and fe
male passengers, and it looks like there were more male passengers than female pas
sengers.\nThe histogram shows the age distribution of the passengers, and we can s
ee that many of them were between 20 and 40 years old.\nOverall, these graphs help
us understand the passenger information in a clear and visual way.\n\n'
```

In [ ]:
```
'''

Ass09
02-03-2026

'''
'''
1. College

Problem: Predicting student performance
Input → Output:
Input can include attendance, assignment marks, study hours, and internal exam scor
Output will be the predicted final grade of the student.

This machine learning system can help teachers understand how students are performi
If the model predicts that a student might get low marks,the teacher can give extra

2.Healthcare

Problem: Disease prediction
Input → Output:
Input can include patient symptoms, age, medical history, and test results.
Output will be the predicted disease or the risk level of the disease.

This type of machine learning system can help doctors identify possible diseases ea
By analyzing patient data, the model can give predictions that support doctors in m

3.Shopping

Problem: Product recommendation system
Input → Output:
Input can include the user’s search history, previous purchases, and browsing behav
Output will be recommended products that the customer may be interested in.

Machine learning can analyze what customers search for or buy online.
Based on this data, the system suggests products that match the customer’s interest
and helps customers find useful products quickly.

4.College Campus (Transportation)

Problem: Predicting bus crowd or traffic near college
Input → Output:
```

```
Input can include time of the day, number of students using the bus, and traffic da
Output will be the predicted crowd level or traffic condition.

A machine learning model can analyze daily travel patterns of students.
Based on the data, it can predict busy times and traffic conditions, which can help

'''
```

In [1]:
```
'''

Ass10
03-03-2026

'''
import pandas as pd
data = {
    "hours":[1,2,3,4,5,6],
    "marks":[36,46,56,66,76,86]
}
df=pd.DataFrame(data)
x=df[["hours"]]
y=df[["marks"]]
print("features:\n",x)
print("label:\n",y)
```

```
features:
    hours
0      1
1      2
2      3
3      4
4      5
5      6
label:
    marks
0     36
1     46
2     56
3     66
4     76
5     86
```

In [4]:
```
'''

Ass11
09-03-2026

'''
import pandas as pd
from sklearn.linear_model import LinearRegression

data = {
    "size": [500, 700, 900, 1100, 1300],
    "price": [100000, 150000, 200000, 250000, 300000]
}
```

```
df = pd.DataFrame(data)

X = df[["size"]]
y = df["price"]

model = LinearRegression()
model.fit(X, y)

new_size = [[1000]]
predicted_price = model.predict(new_size)

print("Predicted price for 1000 sq ft house:", predicted_price[0])
```

```
Predicted price for 1000 sq ft house: 225000.0
```

```
C:\Users\DELL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarn
ing: X does not have valid feature names, but LinearRegression was fitted with featu
re names
  warnings.warn(
```

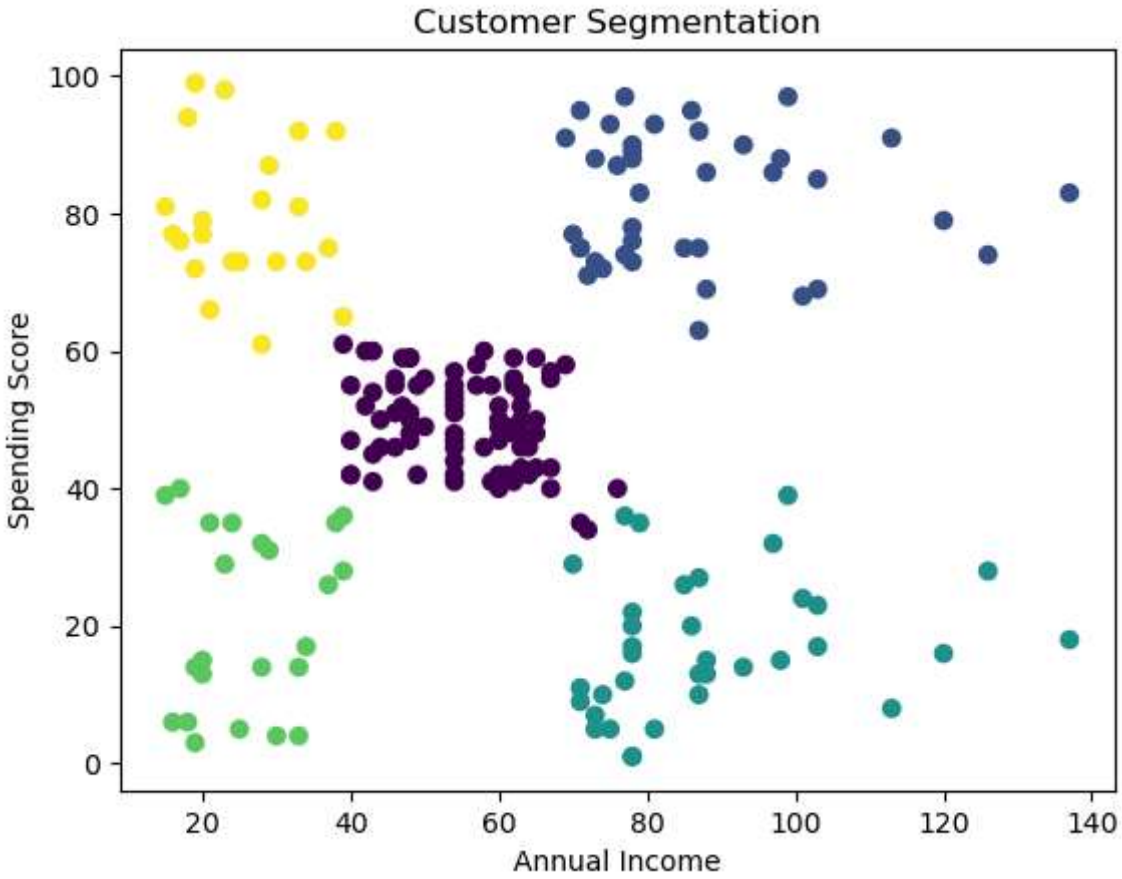
In [ ]:

```
'''

ASS12
10-03-2026

'''

'''
SPAM CLASSIFIER THINKING

1. Features (What the model looks at)

Some important features to detect spam messages are:

a)Presence of words like “free”, “win”, “offer”
b)Number of links in the message
c)Length of the message
d)Use of capital letters (like ALL CAPS)
e)Sender email or unknown number

2.Data Needed

To build a spam detection system, we need a dataset that contains:

a)Many spam messages
b)Many normal messages
c)Labels showing whether each message is spam or not


3.Possible Mistakes

The system may make some errors:

a)False Positive: A normal message is marked as spam
b)False Negative: A spam message is not detected
```

In [ ]:
```
'''
ass13
07-03-2026

KNN in Real life

'''

'''
The K-Nearest Neighbors (KNN) algorithm works by finding users with similar interes
In apps like Netflix, the system compares what a user has watched with other users.
If two users have similar watching patterns, they are considered “neighbors”.
Based on this, the system recommends movies or shows that similar users liked.
'''

'''

User    Movie A Movie B Movie C
User1     5             4         1
User2     5             5         2
User3     1             2         5

Here, User1 and User2 have given similar ratings to Movie A and Movie B. So, they a
User3 has very different ratings, so it is not similar to User1.
If User2 liked a movie that User1 has not watched, the system may recommend that mo

KNN works by finding the most similar users and using their preferences to make rec
This helps platforms like Netflix suggest content that matches the user’s interests

'''
```

In [5]:
```
'''
ass14
11-03-2026

'''
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df = pd.read_csv("Mall_Customers.csv")

X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

kmeans = KMeans(n_clusters=5, random_state=0)
df["Cluster"] = kmeans.fit_predict(X)

plt.scatter(X.iloc[:,0], X.iloc[:,1], c=df["Cluster"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Segmentation")
plt.show()
```

```
C:\Users\DELL\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarni
ng: KMeans is known to have a memory leak on Windows with MKL, when there are less c
hunks than available threads. You can avoid it by setting the environment variable O
MP_NUM_THREADS=1.
  warnings.warn(
```



In [ ]:

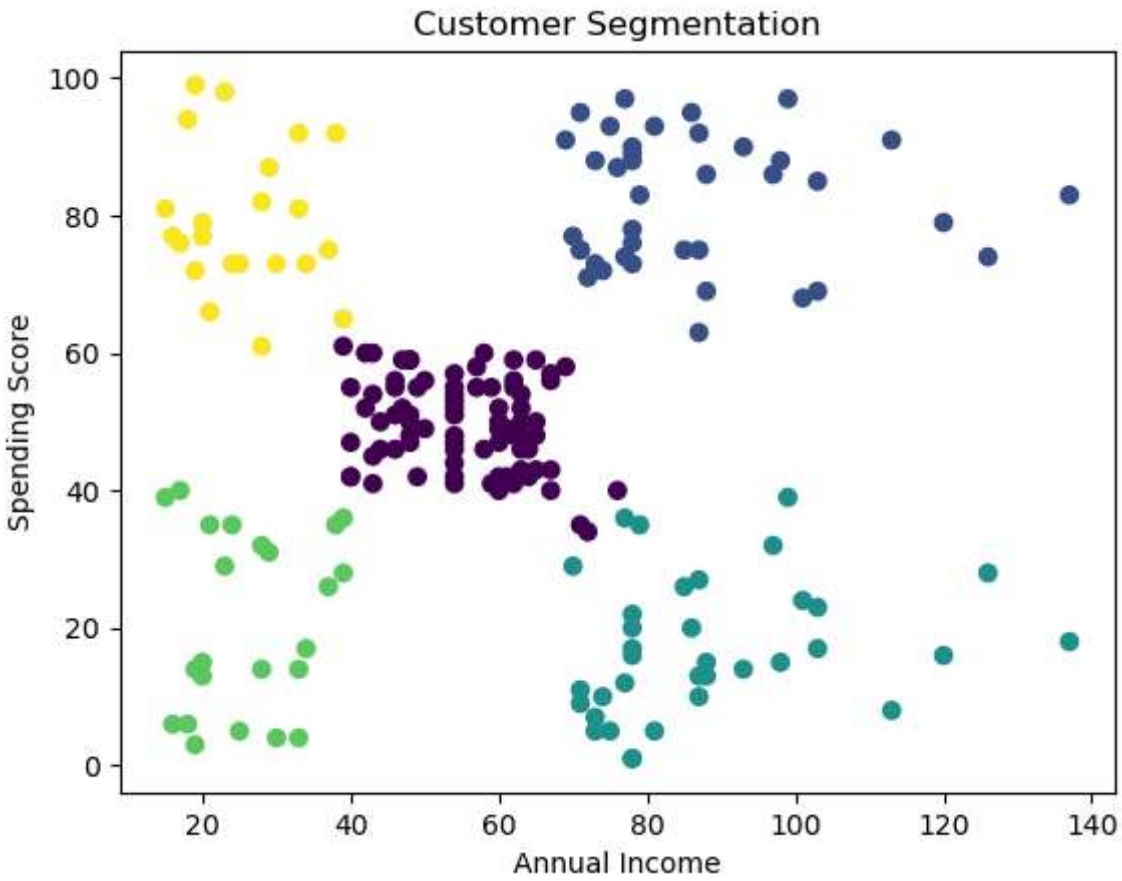
```
'''

ass15
12-03-2026

'''

'''

            Is it Sunny?
            /         \
         Yes           No
        /               \
   Is it Hot?        Play Outside
     /     \
   Yes      No
  /           \
Do Not     Is it Windy?
Play        /      \
           Yes      No
         Do Not    Play
          Play     Outside
```

```
This decision tree helps us decide whether we should play outside or not.
First, it checks if the weather is sunny.
If it is not sunny, we can directly play outside.
If it is sunny, then it checks if it is too hot. If it is hot, we should not play.
If it is not hot, then it checks if it is windy.
If it is windy, we should not play, otherwise we can play outside.

Decision trees make decisions step by step using conditions.
They are easy to understand and help in making simple predictions based on differen

'''
```

In [6]:

```
'''

ass16
18-03-2026

'''

import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df = pd.read_csv("Mall_Customers.csv")

X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

kmeans = KMeans(n_clusters=5, random_state=0)
df["Cluster"] = kmeans.fit_predict(X)

plt.scatter(X.iloc[:,0], X.iloc[:,1], c=df["Cluster"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Segmentation")
plt.show()
```

```
C:\Users\DELL\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarni
ng: KMeans is known to have a memory leak on Windows with MKL, when there are less c
hunks than available threads. You can avoid it by setting the environment variable O
MP_NUM_THREADS=1.
  warnings.warn(
```



In [ ]:
```
'''

Customer Groups

High income – high spending
High income – low spending
Low income – high spending
Low income – low spending
Average customers

'''
```

In [ ]:
```
'''

ass17
20-03-2026

'''
'''
Text Challenges
'''

'''

hey bro!!! how r u 😄
omg this is sooo coool!!!
i cant belive this happend
lol 😂😂 that was funny
```

```
gud mrng hav a nice day
wat r u doing??
this is amazng!!!
i luv this song ❤️
plz send me the file asap
thx for ur help!!!
y r u late??
this movie is lit 🔥
im sooo tired rn
cant wait 4 the weekend
u r the best!!
dis is not gud
omg!!! unbelievable 😱
c u soon
that was gr8
im happy 😊
Issues Found
Slang: bro, lol, omg, gr8
Emojis: 😂 ❤️ 🔥 😱 😊
Typos: belive, happend, amazng
Short forms: u, r, plz, thx


Preprocessing Needed

To clean the text:
1. Convert all text to lowercase
2. Remove punctuation and emojis
3. Correct spelling mistakes
4. Replace short forms (u → you)
5. Remove unnecessary words (stopwords)


'''
```

In [11]:

```
'''

ass18
21-03-2026

'''

import pandas as pd
import string
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')

text = "Hey bro!!! How are you doing today? 😊"

text = text.lower()
text = text.translate(str.maketrans('', '', string.punctuation))

stop_words = set(stopwords.words('english'))
words = text.split()
```

```
cleaned_words = [word for word in words if word not in stop_words]
cleaned_text = " ".join(cleaned_words)

print("Cleaned Text:", cleaned_text)
```

```
Cleaned Text: hey bro today 😊
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [ ]: